

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 12
“SEARCHING”



DISUSUN OLEH:
Dharma Chandra Viriya
109082500052
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

CONTOH SOAL

1. Latihan1

Source Code:

Output:

Deskripsi Program:

SOAL LATIHAN

Statement pencarian

1. Latihan1

Source Code:

```
package main

import "fmt"

func main() {
    var arr = [21]int{}
    totalVote := 0
    validVote := 0

    for {
        var suara int
        fmt.Scan(&suara)

        if suara == 0 {
            break
        }

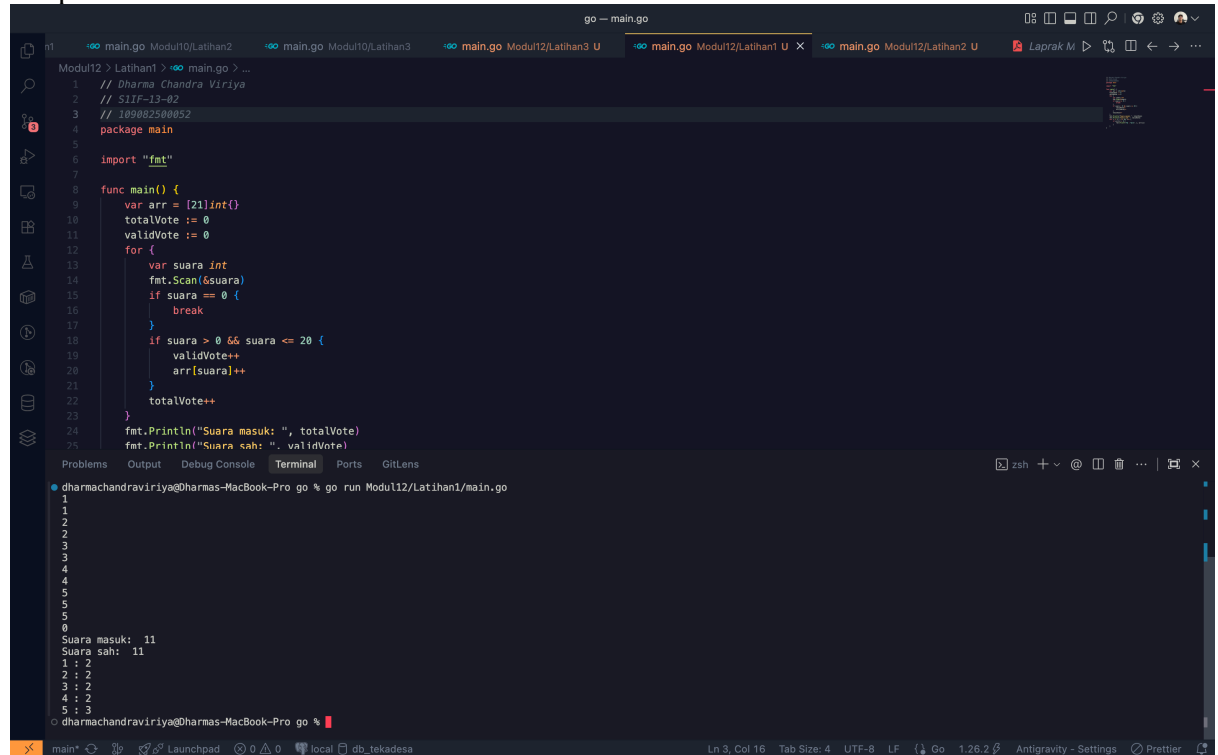
        if suara > 0 && suara <= 20 {
            validVote++
            arr[suara]++
        }

        totalVote++
    }

    fmt.Println("Suara masuk: ", totalVote)
    fmt.Println("Suara sah: ", validVote)

    for i := 1; i <= 20; i++ {
        if arr[i] > 0 {
            fmt.Printf("%d : %d\n", i, arr[i])
        }
    }
}
```

Output:



```
go -- main.go
Modul12 > Lathant1 > go main.go > ...
1 // Dharma Chandra Viriya
2 // SIIP-13-02
3 // 10500250052
4 package main
5
6 import "fmt"
7
8 func main() {
9     var arr = [21]int{}
10    totalVote := 0
11    validVote := 0
12    for {
13        var suara int
14        fmt.Scan(&suara)
15        if suara == 0 {
16            break
17        }
18        if suara > 0 && suara <= 20 {
19            validVote++
20            arr[suara]++
21        }
22        totalVote++
23    }
24    fmt.Println("Suara masuk: ", totalVote)
25    fmt.Println("Suara sah: ", validVote)
26}

Problems Output Debug Console Terminal Ports GitLens
dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul12/Lathant1/main.go
1
2
3
4
5
6
Suara masuk: 11
Suara sah: 11
1 : 2
2 : 2
3 : 2
4 : 2
5 : 3
dharmachandraviriya@Dharmas-MacBook-Pro go %
```

Deskripsi Program:

Program ini dibuat untuk membantu proses penghitungan suara pada pemilihan ketua RT yang memiliki 20 calon. Program menerima masukan berupa sejumlah nomor calon yang dipilih oleh warga. Data suara dibaca satu per satu hingga ditemukan angka `0` yang menandakan akhir masukan. Selain menghitung jumlah seluruh suara yang masuk, program juga melakukan validasi untuk memastikan bahwa nomor calon yang dipilih berada pada rentang 1 sampai 20. Suara yang berada di luar rentang tersebut dianggap tidak sah dan tidak dihitung sebagai suara valid. Setelah seluruh data selesai dibaca, program menampilkan jumlah suara yang masuk, jumlah suara sah, serta daftar calon yang memperoleh suara beserta jumlah suara yang diterimanya. Secara teknis, program menggunakan array `arr` berukuran 21 elemen untuk menyimpan frekuensi suara masing-masing calon, dengan indeks 1 sampai 20 digunakan sebagai nomor calon. Variabel `totalVote` digunakan untuk menghitung seluruh data suara yang masuk, baik valid maupun tidak valid, sedangkan `validVote` digunakan untuk menghitung jumlah suara yang sah. Proses pembacaan data dilakukan menggunakan perulangan tak hingga (`for`) yang akan berhenti ketika pengguna memasukkan nilai `0`. Setiap suara yang bernilai antara 1 hingga 20 akan menambah nilai `validVote` dan elemen array yang sesuai akan ditambah satu untuk mencatat suara calon tersebut. Setelah proses input selesai, program menampilkan jumlah suara masuk dan jumlah suara sah. Selanjutnya, program menelusuri array dari indeks 1 hingga 20 dan mencetak hanya calon yang memiliki jumlah suara lebih dari nol. Dengan memanfaatkan array sebagai pencacah frekuensi, proses penghitungan suara dapat dilakukan secara efisien dalam satu kali pembacaan data tanpa perlu melakukan pencarian atau pengurutan tambahan.

2. Latihan2

Source Code:

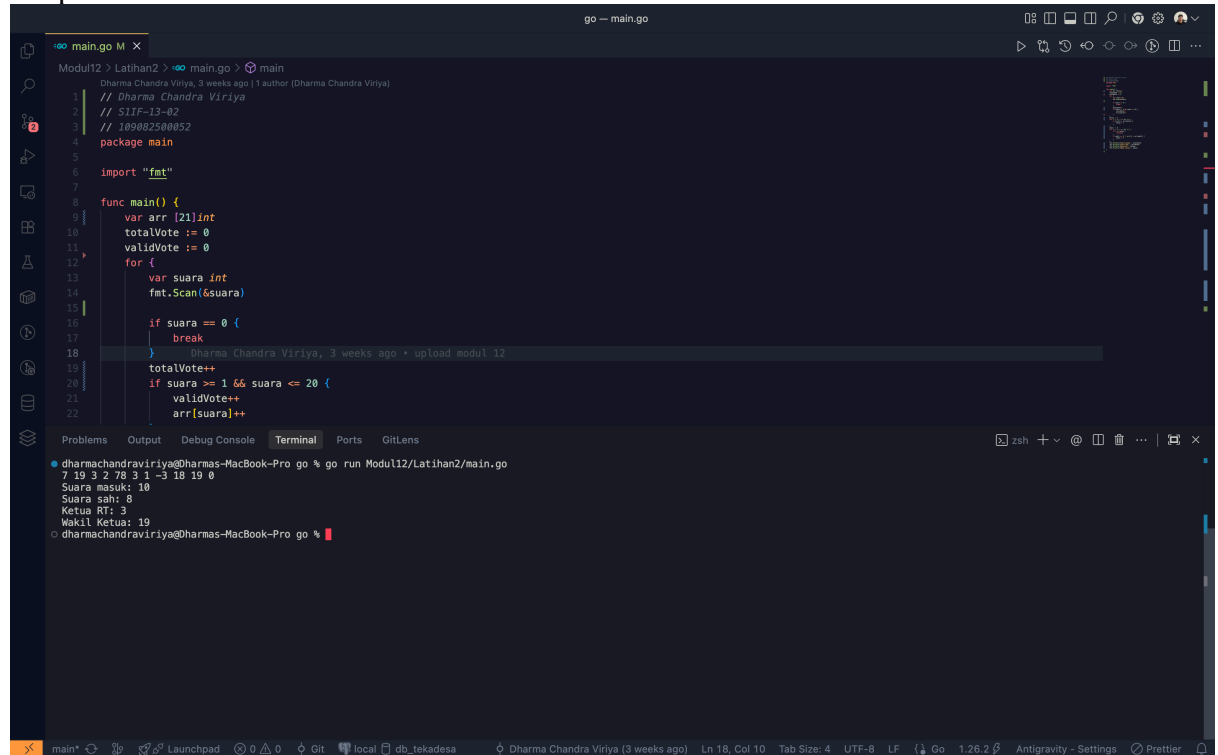
```
package main

import "fmt"

func main() {
    var arr [21]int
    totalVote := 0
    validVote := 0
    for {
        var suara int
        fmt.Scan(&suara)

        if suara == 0 {
            break
        }
        totalVote++
        if suara >= 1 && suara <= 20 {
            validVote++
            arr[suara]++
        }
    }
    ketua := 1
    for i := 2; i <= 20; i++ {
        if arr[i] > arr[ketua] {
            ketua = i
        }
    }
    wakil := 0
    for i := 1; i <= 20; i++ {
        if i == ketua {
            continue
        }
        if wakil == 0 || arr[i] > arr[wakil] {
            wakil = i
        }
    }
    fmt.Println("Suara masuk:", totalVote)
    fmt.Println("Suara sah:", validVote)
    fmt.Println("Ketua RT:", ketua)
    fmt.Println("Wakil Ketua:", wakil)
}
```

Output:



The screenshot shows a Go IDE with a file named `main.go`. The code defines a `main` function that reads input votes until a 0 is entered. It uses an array `arr` to store valid votes (1-20) and variables `totalVote` and `validVote` to track the counts. The program then finds the highest frequency in the array to determine the winning candidate.

```
1 // Dharma Chandra Viriya, 3 weeks ago | 1 author (Dharma Chandra Viriya)
2 // S1IF-13-02
3 // 100082500052
4 package main
5
6 import "fmt"
7
8 func main() {
9     var arr [21]int
10    totalVote := 0
11    validVote := 0
12    for {
13        var suara int
14        fmt.Scan(&suara)
15
16        if suara == 0 {
17            break
18        }
19        totalVote++
20        if suara >= 1 && suara <= 20 {
21            validVote++
22            arr[suara]++
23        }
24    }
25
26    // Find the candidate with the highest frequency
27    maxVote := 0
28    maxCandidate := 0
29    for i := 1; i < len(arr); i++ {
30        if arr[i] > maxVote {
31            maxVote = arr[i]
32            maxCandidate = i
33        }
34    }
35
36    fmt.Println("Suara masuk:", totalVote)
37    fmt.Println("Suara sah:", validVote)
38    fmt.Println("Ketua RT:", maxCandidate)
39    fmt.Println("Wakil Ketua:", maxCandidate-1)
40}
```

The terminal output shows the execution of the program with the following input and output:

```
dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul12/Latihan2/main.go
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil Ketua: 19
dharmachandraviriya@Dharmas-MacBook-Pro go %
```

Deskripsi Program:

Program ini digunakan untuk menentukan pasangan ketua dan wakil ketua RT berdasarkan hasil pemungutan suara. Data suara dibaca secara berulang hingga ditemukan nilai 0 sebagai penanda akhir input, kemudian setiap suara yang valid (bernilai 1–20) dihitung frekuensinya menggunakan array `arr`, sementara jumlah seluruh suara masuk dan suara sah dicatat pada variabel `totalVote` dan `validVote`. Setelah proses input selesai, program melakukan pencarian calon ketua dengan frekuensi suara tertinggi melalui iterasi pada array, kemudian melakukan iterasi kedua untuk menentukan wakil ketua dengan mengabaikan calon yang telah terpilih sebagai ketua. Terakhir, program menampilkan jumlah suara masuk, jumlah suara sah, serta nomor calon yang terpilih sebagai ketua dan wakil ketua RT.

3. Latihan3

Source Code:

```
package main

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func isiArray(n int) {
    /* I.S. terdefinisi integer n, dan sejumlah n data
       sudah siap pada piranti masukan.
       F.S. array data berisi n bilangan yang dibaca
       dari piranti masukan. */

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    /* mengembalikan posisi k dalam array data dengan n elemen.
       Posisi dimulai dari 0. Jika tidak ada, kembalikan -1. */

    left := 0
    right := n - 1

    for left <= right {
        mid := (left + right) / 2

        if data[mid] == k {
            return mid
        } else if data[mid] < k {
            left = mid + 1
        } else {
            right = mid - 1
        }
    }

    return -1
}

func main() {
    /* I.S. terdefinisi bilangan bulat positif n dan k
       pada piranti masukan, serta n buah data integer
       positif yang sudah terurut membesar.
       F.S. tercetak posisi data k dalam array jika ada,
```


atau "TIDAK ADA" jika k tidak ditemukan. */

var n, k int

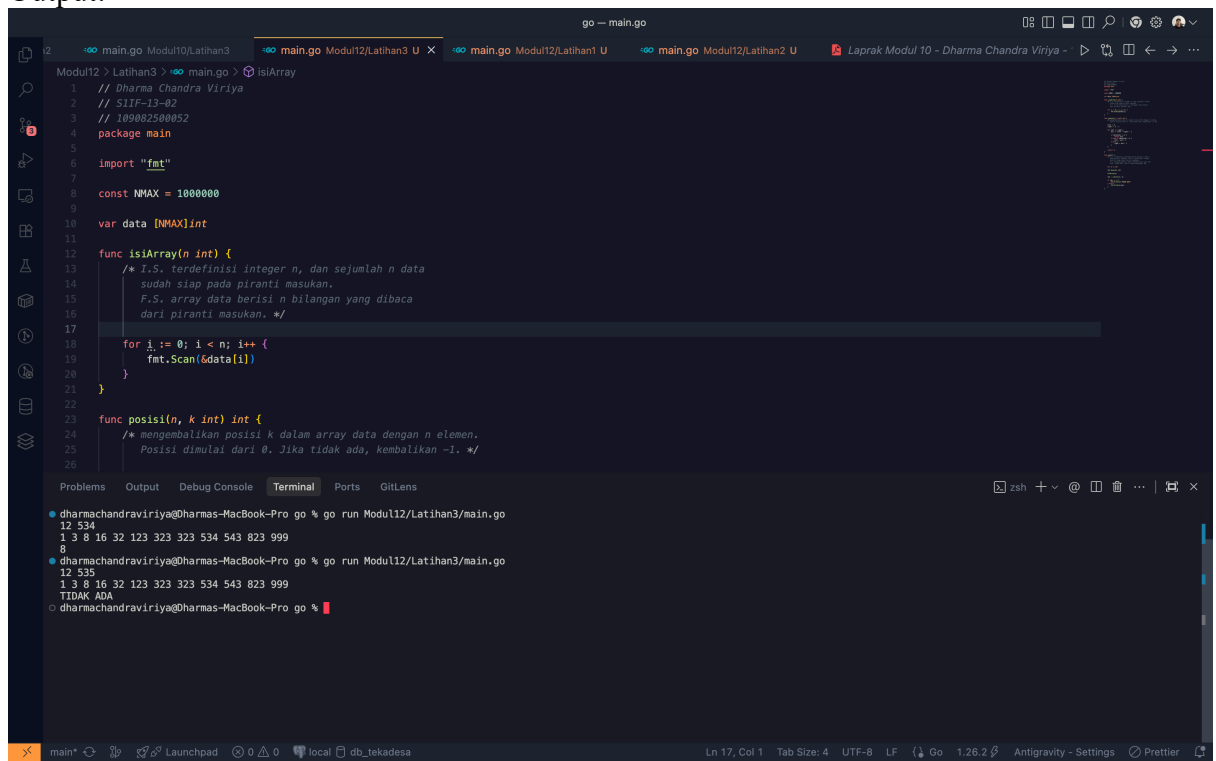
fmt.Scan(&n, &k)

isiArray(n)

idx := posisi(n, k)

```
if idx == -1 {  
    fmt.Println("TIDAK ADA")  
} else {  
    fmt.Println(idx)  
}  
}
```

Output:



The screenshot shows a Go IDE with a file named `main.go` in the `Modul12/Latihan3` directory. The code implements the `isiArray` and `posisi` functions. The `isiArray` function takes an integer `n` and fills an array `data` with values from 1 to `n`. The `posisi` function takes an integer `n` and a value `k`, and returns the index of `k` in the array, or -1 if not found. The terminal shows the output of running the program twice. In the first run, the input is `12 534` and the output is `1 3 8 16 32 123 323 323 534 543 823 999`. In the second run, the input is `12 535` and the output is `1 3 8 16 32 123 323 323 534 543 823 999` followed by `TIDAK ADA`.

```
1 // Dharma Chandra Viriya  
2 // S1IF-13-02  
3 // 100082500852  
4 package main  
5  
6 import "fmt"  
7  
8 const NMAX = 1000000  
9  
10 var data [NMAX]int  
11  
12 func isiArray(n int) {  
13     /* I.S. terdefinisi integer n, dan sejumlah n data  
14     sudah siap pada piranti masukan.  
15     F.S. array data berisi n bilangan yang dibaca  
16     dari piranti masukan. */  
17     for i := 0; i < n; i++ {  
18         fmt.Scan(&data[i])  
19     }  
20 }  
21  
22 func posisi(n, k int) int {  
23     /* mengembalikan posisi k dalam array data dengan n elemen.  
24     Posisi dimulai dari 0. Jika tidak ada, kembalikan -1. */  
25     for i := 0; i < n; i++ {  
26         if data[i] == k {  
27             return i  
28         }  
29     }  
30     return -1  
31 }  
32  
33 func main() {  
34     isiArray(12)  
35     fmt.Scan(&n, &k)  
36     idx := posisi(n, k)  
37     if idx == -1 {  
38         fmt.Println("TIDAK ADA")  
39     } else {  
40         fmt.Println(idx)  
41     }  
42 }
```

Terminal Output:

```
dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul12/Latihan3/main.go  
12 534  
1 3 8 16 32 123 323 323 534 543 823 999  
8  
dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul12/Latihan3/main.go  
12 535  
1 3 8 16 32 123 323 323 534 543 823 999  
TIDAK ADA  
dharmachandraviriya@Dharmas-MacBook-Pro go %
```

Deskripsi Program:

Program ini dibuat untuk mencari posisi suatu bilangan dalam kumpulan data yang telah terurut secara menaik. Pengguna memasukkan banyaknya data (n), nilai yang ingin dicari (k), serta sejumlah n bilangan bulat yang sudah berada dalam keadaan terurut. Program kemudian melakukan pencarian terhadap nilai k di dalam array dan menampilkan posisi indeksinya apabila ditemukan. Jika nilai tersebut tidak terdapat dalam array, program akan menampilkan tulisan **"TIDAK ADA"** sebagai keluaran. Secara teknis, program menggunakan array global `data` dengan kapasitas maksimum satu juta elemen untuk menyimpan seluruh data masukan. Proses pengisian array dilakukan oleh prosedur `isiArray`, yang membaca sebanyak n bilangan dari piranti masukan dan menyimpannya ke dalam array. Pencarian data dilakukan oleh fungsi `posisi` menggunakan algoritma **Binary Search**. Algoritma ini bekerja dengan membandingkan nilai yang dicari dengan elemen tengah array, kemudian mempersempit ruang pencarian ke bagian kiri atau kanan array sesuai hasil perbandingan. Proses tersebut dilakukan berulang hingga data ditemukan atau ruang pencarian habis. Jika data ditemukan, fungsi mengembalikan indeks posisi data dalam array yang dimulai dari 0. Sebaliknya, jika data tidak ditemukan, fungsi mengembalikan nilai `-1`. Pada fungsi `main`, hasil pencarian diperiksa dan ditampilkan kepada pengguna, baik berupa posisi data maupun pesan bahwa data tidak ditemukan. Penggunaan algoritma Binary Search membuat proses pencarian menjadi lebih efisien dibandingkan pencarian linear karena hanya memerlukan kompleksitas waktu **$O(\log n)$** pada data yang sudah terurut.

DAFTAR PUSTAKA